

TensorNet Subpackage: Usage and Coding Documentation

© Copyright 2023, 2025 Anthony D. Dutoi

This file is part of **Qode**.

Qode is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Qode is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with **Qode**. If not, see <http://www.gnu.org/licenses/>.

1 Quick Start

The following code is imported directly from `tensornet/tests/tutorial.py`. It is in the `tests/` directory because running it and checking the output according to the embedded comments serves as a quick (but not exhaustive) test that the subpackage is working correctly. It is named `tutorial.py` because it is heavily commented to illustrate basic usage.

```
1 # (C) Copyright 2023, 2025 Anthony D. Dutoi
2 #
3 # This file is part of Qode.
4 #
5 # Qode is free software: you can redistribute it and/or modify
6 # it under the terms of the GNU General Public License as published by
7 # the Free Software Foundation, either version 3 of the License, or
8 # (at your option) any later version.
9 #
10 # Qode is distributed in the hope that it will be useful,
11 # but WITHOUT ANY WARRANTY; without even the implied warranty of
12 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
13 # GNU General Public License for more details.
14 #
15 # You should have received a copy of the GNU General Public License
16 # along with Qode. If not, see <http://www.gnu.org/licenses/>.
17 #
18
19 ### If everything is working, running this should print out a bunch of
20 ### random numbers that are < ~1e-15, and a single instance of a warning
21 ### about extracting individual elements. The usage is explained in the
22 ### comments below.
23
24 import sys
```

```

25 import math
26 import numpy
27 #import torch
28 import tensorly
29 from qode.math.tensornet import np_tensor, tl_tensor, evaluate, increment,
    raw, scalar_value
30
31 p,q,r,s = 'pqrs'          # lower the number of quotes we need to type
32
33 #####
34 ### In tensornet, backend is a tensor-local concept, but this file does not
35 ### make use of multiple backends, so here is an example of how one makes
36 ### backend a "global" choice (by wrapping the user functions that deal
37 ### with raw tensors).
38
39 #tensorly.set_backend("pytorch")    # use tensorly with pytorch
40
41 # use numpy directly as backend
42 #zeros, einsum, prim_tensor =      numpy.zeros,      numpy.einsum, np_tensor
43 # use tensorly backend (numpy or pytorch)
44 zeros, einsum, prim_tensor = tensorly.zeros, tensorly.einsum, tl_tensor
45
46 # package up the repetitive logic of checking accuracy
47 def check(xpt, ref):
48     # numpy
49     #return numpy.linalg.norm(xpt - ref) / numpy.linalg.norm(ref)
50     # tensorly (BTW, what tensorly calls \|ell^2 is Frobenius)
51     return tensorly.norm(xpt - ref) / tensorly.norm(ref)
52
53 def random_tensor(*shape):
54     # numpy
55     #return numpy.random.random(shape)
56     # tensorly numpy (default)
57     return tensorly.tensor(numpy.random.random(shape))
58     # tensorly pytorch
59     #return tensorly.tensor(torch.rand(*shape))
60
61 #####
62
63
64
65     ### The Basics
66
67 # Some original "raw" tensors
68 A_ = random_tensor(10, 10)
69 B_ = random_tensor(10, 8)
70 C_ = random_tensor(8, 10)
71 D_ = random_tensor(8, 8)
72 U_ = random_tensor(8)
73 V_ = random_tensor(10)
74
75 # Prepare "primitive" tensors for tensornet using wrapping function for raw
76 # arrays. The original data is never modified by tensornet calls and
77 # functions.
78 A = prim_tensor.init(A_)
79 B = prim_tensor.init(B_)

```

```

80 C = prim_tensor.init(C_)
81 D = prim_tensor.init(D_)
82 U = prim_tensor.init(U_)
83 V = prim_tensor.init(V_)
84
85 # - The top-level utility is this contract function. It does no computation
86 #   at this point, but successive calls build up a tensor "network" of
87 #   contractions.
88 # - Each argument to contract is generated by a call on the input tensor
89 #   (primitive or network), whose own arguments are either (1) a label whose
90 #   recurrence specifies indices to be contracted, or (2) an integer whose
91 #   order specifies the
92 #   permutation of the free indices in the contracted product.
93 # - Contract will raise an exception if a logical error occurs with the
94 #   shapes, like contracting indices of incompatible lengths.
95 # - There is no limit on the number of arguments and (to within machine
96 #   precision) the result will not depend in any way on how they are ordered
97 #   (not even efficiency).
98 # - Scalar arguments can also be given in place of tensors, which multiply
99 #   the result.
100 # - A contraction "label" (often a letter) can be any hashable object that
101 #   is not an integer.
102 AB1      = A(0,p) @ B(p,1)
103 CD1      = C(p,1) @ D(0,p)
104 ABCD1    = AB1(0,p) @ CD1(p,1)
105 ABCD1 *= 2
106
107 # The above can also be done by using the @ operator to separate the input
108 # tensors. Just using the call "()" notation on a tensor already eventually
109 # implies that the contract function will be called, and the @ operator
110 # collects arguments for this. This is encouraged for easy reading, but it
111 # is the least well tested part of the module (so maybe check against
112 # explicit contract() if you suspect a problem). More will be said below,
113 # but one side effect of this syntax is that the range checking is postponed
114 # because the calling of contract() is delayed ("lazy").
115 AB2      = A(0,p) @ B(p,1)
116 CD2      = C(p,1) @ D(0,p)
117 ABCD2    = AB2(0,p) @ CD2(p,1)
118 ABCD2 *= 2
119
120 # Let us test the above. The evaluate() function (more on this further
121 # below) will cause the actual numerical contractions to be performed (in an
122 # optimized order) returning a new primitive tensor containing the result.
123 # The user can use evaluate() to force the computations of intermediates
124 # that might be used in multiple further expressions, for example. The
125 # raw() function immediately below calls evaluate() and subsequently returns
126 # the result as a raw tensor, which we can test against by contracting the
127 # original data.
128 test = 2 * einsum("pr,rt,sq,ts->pq", A_, B_, C_, D_)
129 print("relative error in check 1:", check(raw(ABCD1), test))
130 print("relative error in check 2:", check(raw(ABCD2), test))
131
132 # We could have also constructed ABCD in once line as. Note that scalar
133 # multiplication uses the familiar * operator.
134 ABCD3 = 2 * A(0,p) @ B(p,q) @ C(r,1) @ D(q,r)
135 print("relative error in check 3:", check(raw(ABCD3), test))

```

```

136
137 # This illustrates that +, - and * all work on tensors networks, and the
138 # call notation "()" can also be used just to permute indices. The result
139 # of a scalar multiplication (which is fast) is another tensor network whose
140 # internal scalar factor is modified. Similarly, the result of the
141 # subtraction is an abstracted structure that keeps track of the operation
142 # but performs no computation until evaluate() or raw() is used.
143 ABCD_p = 3*ABCD1 - 3*ABCD1(1,0)
144 test = 3*test - 6*einsum("pr,rt,sq,ts->qp", A_, B_, C_, D_)
145 print("relative error in check 4:", check(raw(ABCD_p), test))
146
147 # This illustrates that there are not many restrictions on the order of the
148 # arguments or the placement of the indices. The redundant "1" means that
149 # the 1 index of the contracted product is resolved by setting these two
150 # equal to each other. Also single as well as multiple (not just double)
151 # occurrences of contraction labels are allowed. Note that the same index
152 # can occur multiply on any given tensor, and that outer products (A shares
153 # no indices with others) are allowed.
154 ABCD3 = A(q,0) @ B(1,p) * 3 @ C(p,1) @ D(p,p)
155 test = 3 * einsum("qr,sp,ps,pp->rs", A_, B_, C_, D_)
156 print("relative error in check 5:", check(raw(ABCD3), test))
157
158 # The real utility of tensornet is for something like this. E represents
159 # 4-index tensor, but its contractions with vectors can be done faster if
160 # done internally as contractions with the factors. The scalar_value()
161 # function extracts the scalar-typed value from a 0-index tensor (like
162 # ndarray.item() in numpy).
163 E = A(0,1) @ D(2,3)
164 VVEUU = scalar_value(E(p,q,r,s) @ V(p) @ V(q) @ U(r) @ U(s))
165 test = einsum("pq,rs,p,q,r,s->", A_, D_, V_, V_, U_, U_).item()
166 print("relative error in check 6:", (VVEUU - test) / test)
167
168 # You can take elements of a primitive tensor, ...
169 print("relative error in check 7:", (A[0,0] - A_[0,0]) / A_[0,0])
170 # ... or of a tensor network. For a network, this will reevaluate
171 # everything for every element requested, so it warns the user the first
172 # time about this being inefficient (maybe use evaluate() on a block and
173 # then take elements instead?). To be clear, tensornet is smart enough not
174 # evaluate the whole tensor to obtain a single element; it first restricts
175 # the relevant indices in the primitives before contraction is done. So, in
176 # this particular case, it actually is not inefficient.
177 print("relative error in check 8:",
178       (E[0,0,0,0] - A_[0,0]*D_[0,0]) / (A_[0,0]*D_[0,0]))
179 # Note that it does not warn you a second time.
180 print("relative error in check 9:",
181       (E[1,1,1,1] - A_[1,1]*D_[1,1]) / (A_[1,1]*D_[1,1]))
182
183 # Slicing also works, ...
184 print("relative error in check 10:", check(raw(A[0, 1:5]), A_[0, 1:5]))
185 # ... within contractions ...
186 AC = A[:, :8](0,p) @ C(p,1)
187 test = einsum("pq,qr->pr", A_[ :, :8], C_)
188 print("relative error in check 11:", check(raw(AC), test))
189 # ... and on tensor networks (where again it first acts to slice the
190 # primitives in the network before evaluation is done.
191 test = einsum("p,q->pq", A_[ :, 0], D_[ :, 1])

```

```

192 print("relative error in check 12:", check(raw(E[:,0,:,1]), test))
193
194
195
196     ### Some Finer Points
197
198 # One thing that is good to be aware of is that, if one contracts a sum
199 # (such as F below), the contractions of the terms are always done first and
200 # then added (most likely desired and far simpler dispatching algorithm) ...
201 F = A + 5*B(0,p)@C(p,1)
202 test = einsum("p,pq,q->", V_, A_, V_) +
203     5 * einsum("p,pq,qr,r->", V_, B_, C_, V_)
204 print("relative error in check 13:", check(raw(V(p) @ F(p,q) @ V(q)), test))
205
206 # ... but the user has control over this by forcing evaluation of the
207 # intermediate first, if desired. The evaluate() function performs all
208 # internal contractions and index reductions and returns a primitive (not a
209 # raw) tensor.
210 print("relative error in check 14:",
211     check(raw(V(p) @ evaluate(F)(p,q) @ V(q)), test))
212
213 # Should one need to loop over terms to build a sum, an empty accumulator
214 # can be started as so:
215 F = prim_tensor.zeros()
216 for term in [A, 5*B(0,p)@C(p,1)]:    # pretend generated by some algorithm
217     F += term
218 print("relative error in check 15:", check(raw(V(p) @ F(p,q) @ V(q)), test))
219
220 # Here is another way to do this, if you know you want a concrete evaluation
221 # immediately. This does not yet save memory with an in-place build, but
222 # the point is to leave the door open for that to be implemented later
223 # (presently just a thin wrapper around +=raw(...)).
224 F_ = zeros(A.shape)
225 for term in [A, 5*B(0,p)@C(p,1)]:
226     increment(F_, term)
227 print("relative error in check 16:",
228     check(einsum("pq,p,q->", F_, V_, V_), test)),
229
230 # And now a comment on a subtlety of the @ operator. By necessity (since
231 # there is no trigger to tell it to stop), it generates "incomplete"
232 # contractions. Consider "ABCD" from above. Done in the way below, the
233 # contraction labels q and r must be shared across two lines. This is a
234 # potential danger since one might not realize a letter has been shared if
235 # lines like these are farther apart. Whereas there might actually be a
236 # niche use case for this behavior it is mostly just a dark corner.
237 ABC4 = A(0,p) @ B(p,q) @ C(r,1)
238 ABCD4 = 2 * ABC4 @ D(q,r)
239 test = 2 * einsum("pr,rt,sq,ts->pq", A_, B_, C_, D_)
240 print("relative error in check 17:", check(raw(ABCD4), test))
241 # However, under normal circumstances, one would not write code in this way,
242 # so the danger is limited. If one really means that, in ABC4, q and r
243 # should be summed over as individual indices, leaving only two free
244 # indices in the product, then future contractions should be written more
245 # like the following (note the call with indices attached to ABC4), where we
246 # have to choose something other than D just for dimension matching.
247 ABCB1 = 2 * ABC4(p,0) @ B(p,1)

```

```

248 test = 2 * einsum("pq,qr,st,pu->tu", A_, B_, C_, B_)
249 print("relative error in check 18:", check(raw(ABCB1), test))
250 # And as a one-liner, we could have
251 ABCB2 = 2 * (A(0,p) @ B(p,q) @ C(r,1))(p,0) @ B(p,1)
252 print("relative error in check 19:", check(raw(ABCB2), test))
253 # And, just for clarity, this is of course the same (in every way) as
254 ABCB3 = 2 * A(s,p) @ B(p,q) @ C(r,0) @ B(s,1)
255 print("relative error in check 20:", check(raw(ABCB3), test))
256
257
258
259     ### About Efficiency
260
261 # Here is a quick test that shows this code can be smarter than einsum in
262 # dispatching contractions.
263 # Setup ...
264 factor = 1
265 if len(sys.argv)==2:
266     factor = float(sys.argv[1])
267 dim = math.floor(factor * 10)
268 M_ = random_tensor(dim, dim, dim, dim)
269 M = prim_tensor.init(M_)
270 T_ = random_tensor(dim, dim)
271 T = prim_tensor.init(T_)
272 # ... and test:
273 TTMTT = raw(M(p,q,r,s) @ T(p,0) @ T(q,1) @ T(r,2) @ T(s,3))
274 print("TensorNet done with 4-index transformation. Waiting on einsum ... ")
275 # See note about opt_einsum in to-do (default in pytorch and accessible with
276 # numpy, also via tensorly)
277 test = einsum("pqrs,pw,qx,ry,sz->wxyz", M_, T_, T_, T_, T_)
278 print("... einsum done.")
279 print("relative error in check 21:", check(TTMTT, test))
280 print("If the timing difference was not dramatic enough, "
281       "run the script with a float argument >~1")
282 print("to scale the dimensions. But do not go nuts. "
283       "The einsum call scales with the 8th power.")
284
285
286
287     ### About Backends
288
289 # Finally, the concept of a backend is local to the tensors. Rather than
290 # have a global setting,
291 # any two tensors with the same backend may enter into tensorNet expressions
292 # with each other.
293 # The long way of doing this for a numpy backend would be as below. The
294 # default is not to make
295 # a copy of the original tensor data, but simply to wrap it. As long as the
296 # user has no intention
297 # of changing the original data (tensorNet will not either), it is more
298 # efficient, but if you
299 # are getting weird results and want to be sure, or you just know you cannot
300 # promise not to
301 # modify the original, then use the copy_data flag. The np_tensor function
302 # used above, is
303 # equivalent to the first option below and is provided for convenience.

```

```

297
298 # from qode.math.tensornet import primitive_tensor_wrapper, numpy_backend
299 # np_tensor_ = primitive_tensor_wrapper(numpy_backend) #
    this ...
300 # np_tensor_ = primitive_tensor_wrapper(numpy_backend, copy_data=True) #
    ... or this
301 # A = np_tensor_(A_)
302
303 # One can see now that providing a substitute for numpy_backend above is all
    one needs for a custom
304 # backend. This should be a module that has the following functions defined
    in it.
305 #
306 # def copy_data(tensor):
307 #     # Returns a tensor referencing independent numerical data that is a
    copy of that of tensor
308 #     # for numpy this just returns numpy.array(tensor)
309 #
310 # def scalar_value(tensor):
311 #     # Assuming the tensor has no indices left, return the internal value as
    a scalar type (like float)
312 #     # For numpy, this just returns tensor.item()
313 #
314 # def scalar_tensor(scalar):
315 #     # Return the scalar in the data type of a 0-index tensor
316 #     # For numpy, this just returns numpy.array(scalar)
317 #
318 # def shape(tensor):
319 #     # Give the shape (dimensions) of tensor as a tuple of integers
320 #     # For numpy, this just returns tensor.shape
321 #
322 # def contract(*tensor_factors):
323 #     # The arguments of this function are exactly of the same format as the
    tensornet contract()
324 #     # function. The difference is that it is likely to be called with a
    much smaller number
325 #     # of arguments, as the tensornet machiney grinds down the contraction
    network stepwise.
326 #     # A user is free to make this as general or as limited as necessary.
    For example, if only
327 #     # pairwise contractions of indices on two distinct tensors is every
    necessary (by-and-large
328 #     # the dominant use case), then this routine will only every be called
    as such. The implementor
329 #     # can just raise a NotImplementedError if a different use case is ever
    given.
330 #     # For numpy, the general use case can be mapped onto the arguments of a
    valid einsum call.
331 #
332 # Additionally, the raw tensor data type should understand the operations +,
    -, *, [], where
333 # addition/subtraction is to other tensors of the same shape, and * means
    with a scalar. The
334 # item operator should be able to handle tuples that include slices.
335

```

```

336 # PS - In this directory, you see a dummy_tensor module, which is supported
      by the dummy_backend
337 # module provided by tensornet. This does not do any math but just prints
      out the contractions
338 # that are being performed. Originally used to debug tensornet, it might
      come in handy for users
339 # to verify that their code is being executed as expected (and possibly find
      problems with the
340 # contraction-order heuristic).

```

2 More Stuff

In here should also be the outline of `tensor_base` and an example of a backend interface class. Also, we should start with some overview of the file/directory structure.

2.1 Original introduction to `tensors.py`

We have 4 different types of top-level tensors `contraction_expression` `tensor_sum` `tensor_network` `primitive_tensor`. All of these are built off of `tensor_base`. The latter three are built off of `resolved_tensor` (which is based on `tensor_base`) indicating that they are completely well-formed. `contraction_expression` behaves like a tensor so long as it is well formed, but it could be incomplete. The basic problem is that, when connecting a bunch of tensors with `@`, it is not possible to automatically discern when the string is finished (ie, when all intended target free indices are present). Only the user can decide when to "close" the string, but we don't want to make the user do this explicitly. Therefore, we allow this incomplete class, whose `*resolution*` is a `tensor_network` (it always resolves to a `tensor_network`). This is triggered by taking another action on it that is not `@` or `*`. A `tensor_sum` can only contain types `primitive_tensor` and `tensor_network`. An attempt to include another `tensor_sum` will cause it to be expanded out as a single sum (and `contraction_expression` will be automatically resolved to `tensor_network`). A `tensor_network` can only contain type `primitive_tensor`, and including one network (or `contraction_expression`) into another network causes explicit expansion into a single network. Taking this together, this means that a `tensor_network` cannot contain a `tensor_sum`. An attempt to put a `tensor_sum` into a `tensor_network` will result in the hierarchy being swapped to a sum of networks. The reason for this is purely practical. At the moment, I cannot think of a way of finding the best path to evaluate a network that contains sums.

2.2 Original introduction to `tensor_network` in `tensors.py`

Barebones theory (written much later after a forensic debug battle). A `tensor_network` object contains two fundamental pieces of information (and other incidental info). One is a list of contractions. Each contraction itself is a list of two-tuples; each two-tuple identifies a tensor and the index of that tensor involved in the contraction. So if a contraction contains two two-tuples, then two indices are contracted with one another, but there might be more for unusual contractions. The ordering of the list of contractions is irrelevant, as is the ordering of the list of two-tuples that defines a given contraction. The other piece of information is a list of free indices of the result, which correspond to uncontracted indices of the tensors in the network. This list is ordered, and each free index is itself a list of two-tuples with the same tensor-index

structure as the two-tuples that define a contraction. In the most usual case, each such list corresponding to a free index will be of length one, but if there is more than one tensor index that corresponds to a single free index it is because those indices are set equal to each other and reduced to a single free index.

2.3 Original introduction to `primitive_tensor_factory` in `__init__.py`

Philosophy: A backend (in the sense of the value of the argument `backend` below) is a class that contains static methods that perform basic operations on objects that represent tensors of a specific type. These will henceforth be called backend tensors. Tensornet then uses the backend functions to manipulate the backend tensors according to what is encoded by its high-level syntax. Objects of the class defined below are factories for the primitive tensornet tensors which each wrap a backend tensor (and never modifies it). An important point is that a given backend can have multiple factories depending on how what a user is doing.

The creation of a tensornet primitive tensor should only take place via a factory (which is why that class name is "hidden" here with the underscore convention) because this is the mechanism by which a backend tensor is associated with its backend functions and the utility for contraction creation. In the most basic case, this creation is done by the `.init()` method (not `__init__`, which creates the factory). The method `.zeros()` is a convenience function for creation of a tensornet primitive storing a new backend tensor (via a backend method) so the user need not create such a zero tensor first and then "wrap" it (`.scalar_tensor()` is similar and was added for completion, but commenting it out essentially documents that we have not yet found a use case ... the necessity for the backend to be able to create such a thing however has to do with obeying expectations about type uniformity).

Finally we come to the arguments `data_wrap` and `data_unwrap`, the method `.data()`, and the idea of multiple factories for the same backend. It is important that tensornet works only with a notion of backend tensors and the functions it is given for them without knowing anything further about their properties. But the user may be working with some abstracted backend tensors. Although they contain, or somehow represent the existence of, a specific concrete tensor (eg, numpy, tensorly, pytorch), they do not have the usual expected behavior (e.g., `[]` might not even get an element). We will call such things meta tensors, it might be very inconvenient for the user to make their in-house meta tensor behave exactly like a concrete tensor, right down to letting a `numpy.ndarray` (etc) be instantiated from it. It will also be very inconvenient if the user has to always use the unbound function `raw()` and then afterward extract the concrete data for cooperation with other linear algebra libraries. The converse is also true that it will be annoyingly repetitive to always create the meta tensor from a concrete tensor from elsewhere, and then pass that to the factory. So, therefore, a factory can be created that takes the functions `data_wrap` and `data_unwrap`. Although their actions are not constrained as such, the idea is that `data_wrap` is a function that takes a concrete tensor and returns an object of the meta tensor type with which backend works, and `data_unwrap` does the opposite, extracting and returning (by reference) the concrete tensor represented (eg, contained within) the meta tensor. The convenience function `.data()` called on a tensornet tensor causes evaluation of the (potential?) network (sum?) and returns directly the concrete tensor associated with the result. By default, `data_wrap` and `data_unwrap` are just pass-throughs, which means that `.data()` is just an alias for the unbound function `raw()`, which forces evaluation and returns a backend tensor (which is typically already a concrete tensor type). Since this automatic wrapping and unwrapping might not always be the desired behavior, more than one factory can be defined, per need.

Addendum 1: There is no requirement that a meta tensor is not itself also a concrete type, and that this concrete type might be initialized from multiple other concrete types, which might not even be different than its own type. So if we interpret the concrete and meta layers to both be tensorly, where the meta layer can be initialized also from numpy or pytorch (etc), then there it is a perfectly clean for the user to provide as `data_wrap` a function that makes a tensorly tensor from some input which may or may not be of tensorly type), and to leave `data_unwrap` as the default pass-through.

Addendum2: One question that can be asked about the design is: why am I giving in a separated backend with functions that perform manipulations on a tensor, rather than just insisting that backend tensors have certain properties, like `+=` (which would also make the insides of tensornet prettier). The answer is that one might have enough serious work to do in the abstracted/meta backend already, without having to define yet more operations that make the thing look almost like a tensor, but not completely. One can imagine various inheritance and virtual method schemes, but one is always going to end up in one of two places: 1. Forcing the user to make a new class for their backend tensors (and then maybe always, even when using "normal" tensors), or 2. automating the creation of the backend tensors with the expected operators in terms of some other/divorced set of instructions. In case 2, we are back where we started from the user point of view, though the insides of tensornet look nicer (for the price of another file/class and layer of abstraction).

Addendum 3: The `__init__()` method used to take a `copy_data` argument which meant that the factory should copy the provided primitive before storing it as an argument for further use. Probably unusual, but a user might do this if they intended on modifying that primitive for some other use while it was still in use by the tensornet code (you never know ...). This can now be done by the user with the `data_wrap` option.

3 To Do

document thing about `scalar_tensor` OR a new tensor class called `scalar_tensor` which is the result of any operation that reduces to a scalar (recognized from shape). the difference between this an the usual scalar type is that it is mutable (which is very important for evaluating `tensor_sum`

implement `__str__` for all tensors check that internal data is copied in the event of trivial network (single `primitive_tensor`) and that scalar is 1

```
1 backend:
2     allow variable field type (could have a name?)
3     backend base class with more things implemented for "normal"
4     backends
5         scalar_value and scalar_tensor = pass
6         raw_check = pass (user can check that input tensors are as
7         expected)
8         increment -> +=
9         mult -> *
10        subscript -> []
11        shape -> .shape
12        str -> str
13        field = float
```

clean up documentation. Some good doc sentences...

The string of tensors to be contracted often consists of more than two, and so it would be incorrect to assume that a complete and valid tensor has been described after every use of the @ operator. Therefore, it is not safe to assume that the use of @ results in something that is a tensor in every way (eg, it may not even have a valid shape, due to missing free indices). By the same token, since the = operator in python only means "bind" (as opposed to, say, C++), there is no way for the user to indicate when a tensor description is "done". (correction, now use `__call__()`) The dark corner here is that the result of several @ operations remembers the names of its contraction indices and if that result is bound to a variable, it could be @-ed with more tensors, inadvertently using the same letters, leading to unexpected results. This is probably not a big deal, however, because this usage pattern is also unorthodox, since, after binding the result of an @ string to a name, that object should itself be subjected to the call operation `()` referencing the indices of the finished product (with no memory of labels used for internal contractions) before being contracted with other tensors.

3.1 Questions about using `opt_einsum`

<https://joss.theoj.org/papers/10.21105/joss.00753>

Clearly there are advantages to the fact that the authors of `opt_einsum` have given much more thought to their heuristics, but it is not immediately available for all backends, and the fact that my heuristics live above the backend and need to know nothing about the hardware to at least make a *good* guess (maybe not the best) still makes it useful. That said, on further study, we might find that `opt_einsum` is just as easily put on top of other backends as is my own code (eg, it already works on top of numpy and is default in pytorch). If that is the case, and we find it is a hands-down winner over my own heuristic, then we can just have `evaluate()` pass *all* of the arguments directly to `opt_einsum`. This might also be left as a user choice.

Update: `opt_einsum` can now be enabled with the tensorly backend by the use of the following two lines, probably at the top-level:

```
tensorly_plugins.use_opt_einsum()
qode.math.tensornet.backend_contract_path(True)
```

See also:

https://tensorly.org/stable/modules/generated/tensorly_plugins.use_opt_einsum.html#tensorly_plugins.use_opt_einsum

<https://optimized-einsum.readthedocs.io/en/stable/install.html#conda>

I'm not a big fan of how this works because, IMO, all decisions about how to contract tensors should be as local as possible. Say, for example, you wanted to merge two codes that had been using different backends for the tensors. It will be a lot more convenient if you don't have to force them to agree on one. This is why my tensors carry knowledge of their backends with them, and the only thing that matters when two tensors meet is that they are the same (which is presumably the case within any given code domain). Now there is this wart, where you insist that all tensors (even those with different backends) are either going to use the frontend or the backend contraction path. Probably this is ok for well over 90 cases, but I would like to clean it up. At the same time though, making the `backend_contract_path` option a feature of a backend is also not really (ie, could carry this option around with the tensors) because it is not a feature of the backend, it is a feature of a contraction within the context of a backend. We certainly do not want to insist that a tensor created with one way of determining the contraction path cannot be contracted with a tensor with a different "opinion" on the way of determining the contraction path. At the moment then, this is the least of evils. At least a user can dynamically

switch back and forth.